Text-to-SQL Models Make Choices Without Saying So [Experiment, Analysis & Benchmark]

Yichen Xu
University of California, Berkeley
Berkeley, CA, USA
yichen_xu@berkeley.edu

Shipei Lin
School of Data Science
The Chinese University of Hong Kong, Shenzhen
Shenzhen, China
225045017@link.cuhk.edu.cn

Chenhao Ma
School of Data Science
The Chinese University of Hong Kong, Shenzhen
Shenzhen, China
machenhao@cuhk.edu.cn

ABSTRACT

A text-to-SQL answer can run, return a plausible number and still compute something other than what the analyst meant. This happens when a request leaves a calculation choice open, such as whether to count rows or entities, and the model settles the choice without saying so. Benchmarks cannot detect this, because they score one reference interpretation even where the question leaves the choice open. This paper introduces SpecBridge, a framework that tests what a model does with such a choice by changing only that choice. A request set holds three requests on the same data, one for each of two interpretations and one that leaves the choice open, and execution scores every answer. SpecBridge builds request sets on real databases from choices mined from benchmark failures. On 204 hand-validated request sets, four models usually compute both interpretations when each is asked for, yet for the open request they return both interpretations in only 15–58% of those sets, and they name the choice in only 38–70% of all sets. When they settle a choice silently, they take the interpretation that benchmark references take by convention, and on BIRD over two in five of the answers scored wrong on such questions return the other interpretation. A method that first decides whether a request leaves a choice open and then answers each interpretation in its own call misses fewer interpretations and shows fewer wrong answers than plain prompting, at 2.6 calls per request. Sampling and disambiguate-then-parse show far more wrong answers, and the decide-then-ask behavior of production assistants leaves about half of all requests without an answer in that turn. Once the user picks an interpretation, keeping the request and adding one clause beats rewriting it, as production assistants do, by 8.5 to 10.5 points.

KEYWORDS

text-to-SQL, database interfaces, evaluation, controlled tests, ambiguity, language models

PVLDB Reference Format:
Yichen Xu, Shipei Lin, and Chenhao Ma. Text-to-SQL Models Make Choices Without Saying So [Experiment, Analysis & Benchmark]. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/Xyc-arch/specbridge-artifact.

1 INTRODUCTION

Analysts increasingly ask for numbers in natural language and get them from a text-to-SQL system, often without reading the SQL. For these users a query that fails is safe, because the error is visible. The dangerous answer is a query that runs and returns a plausible number, computed differently than the analyst meant. Outside SQL, such small calculation choices have had large consequences. A finding that high public debt slows growth depended on averaging within each country before averaging across countries [9], and a health-care algorithm that measured need by cost cut the Black patients chosen for extra care by more than half [21]. A text-to-SQL system makes such choices on every request, and its user sees only the result. We call such a choice, settled without telling the user, a silent choice. The answer is correct for one interpretation, so checks that look for wrong SQL [2, 4] do not catch it.

Take "the average rental price per day" for each film rating (Figure 1). It can be computed two ways, which we call interpretations. Interpretation A averages each film's daily price, and interpretation B divides the average price by the average rental length. For two films priced at \$6 for two days and \$6 for six days, the first interpretation gives \$2.00 per day and the second \$1.50. Both are simple SQL, and on the `sakila_1` database the two interpretations differ for every rating. For the request as worded, which leaves this choice open, all four models we test, Claude Haiku 4.5, Claude Sonnet 5.5, GPT-6 Sol and GPT-6 Luna (Haiku, Sonnet, GPT-Sol and GPT-Luna below), return A. Sonnet and GPT-Luna say that there is a choice, but only Sonnet also gives B, the ratio of two averages.

A reliable text-to-SQL system must compute the interpretation that a specific request asks for, report a choice that a request leaves open together with the other interpretation, and stay quiet about a choice that the wording already settles. Benchmarks such as Spider and BIRD measure only the first. They score each question against one reference interpretation, even where the question leaves the choice open, so a model that guesses the authors' interpretation scores as well as one that reports the choice. On 79 BIRD questions whose answer the choice changes, accepting both interpretations raises accuracy from 25% to 57% (Section 6.2), so part of the gap between the best BIRD score, 83%, and human performance, 93% [17][1], is not a failure to write SQL. Production assistants handle such

[1] https://bird-bench.github.io

Request on sakila_1: "For each film rating, what is the average rental price per day?"

read as A

Interpretation A: mean of daily prices

```
SELECT rating,
AVG(rental_rate /
    rental_duration)
FROM film GROUP BY rating
```

→ G 0.668, PG 0.642, R 0.662

read as B

Interpretation B: ratio of two averages

```
SELECT rating,
AVG(rental_rate) /
AVG(rental_duration)
FROM film GROUP BY rating
```

→ G 0.597, PG 0.600, R 0.616

Request set (shortened) and the answers counted correct

specific, A	"the average of each film's daily price"	A
specific, B	"average price divided by average duration"	B
open	"average rental price per day"	A or B, choice named

Four models' responses to the open request

	Haiku	Sonnet	GPT-Sol	GPT-Luna
returns A	✓	✓	✓	✓
names the choice	✗	✓	✗	✓
also returns B	✗	✓	✗	✗

Figure 1: One request, two correct interpretations (shaded: the SQL that differs), and four models' responses. Each specific request asks for one interpretation; the open request keeps the user's original wording unchanged.

Table 1: SpecBridge and the two closest benchmarks of ambiguous text-to-SQL. Schema: synonym or split columns and tables; wording: scope, attachment, vagueness; calculation: ties, counting units, missing values, aggregation level. Mined: in our library of choices from benchmark failures, but not built into the request sets we test here.

	AmbiQT [3]	AMBROSIA [25]	SpecBridge
Choices in the schema	✓	×	mined
Choices in the wording	×	✓	mined
Choices in the calculation	×	×	✓
Benchmark databases, unmodified	×	×	✓
Measures reference conventions	×	×	✓
One request per interpretation	×	✓	✓
Separates computing from reporting	×	×	✓
Checks the stated interpretation	×	×	✓
Penalizes raising a settled choice	×	×	✓

choices by stopping: Snowflake's Cortex Analyst rejects a question its classifier marks ambiguous [28], Databricks' Genie asks a clarifying question where its authors' instructions say so [5], and Microsoft's Fabric data agent leaves it to instructions its authors write [19]. Asking needs the same first step, noticing that a choice exists, and costs a round trip on every flagged request. Ambiguity benchmarks [3, 25] score whether a model's outputs contain all interpretations of an ambiguous question. They cannot tell a model that cannot compute an interpretation from one that computes it but stays silent, nor penalize raising a settled choice (Table 1).

Our key idea is to change only the choice. For each test we write three requests on the same database: a specific request for interpretation A, a specific request for interpretation B, and the open request, which leaves the choice open, so the request is ambiguous. With the two reference queries they form a request set. Because the three requests differ only in the choice, each failure has one cause. The model cannot compute an interpretation, computes one but picks the other, or computes both but does not tell the user. For the open request, the model is asked to state its assumptions and may give SQL for other interpretations; offering the other interpretation for a specific request is a false alarm, as the wording settles it.

Building request sets at scale is hard for three reasons. First, a test must control what each query computes and describe it exactly, on databases that nobody has annotated. Second, the choices must be ones that matter in real requests, not invented ambiguities. Third, the three requests must differ only in the choice, yet a rewritten request easily changes other requirements. SpecBridge addresses each in turn. It generates SQL from a catalogue of column meanings under rules on column types, and code describes each query exactly in words (Section 2.2). It mines choices from BIRD and Spider questions that models fail as worded but answer correctly from an exact description of the reference SQL, and keeps only edits that change the answer on the data (Section 2.3). It writes the three requests from one list of shared requirements, and code and two reviewer models check every set. Two authors then cross-checked all 307 passing sets, blind to model results, and kept 232 (76%); results use the 204 of them on choice types where real references disagree, so none rests on model reviewers alone (Sections 2.4 and 4.2). Execution, not a model, decides every accuracy score in this paper.

Finding 1: Models make choices without saying so (Section 5). On 204 accepted request sets (Figure 6), the models answer both specific requests correctly in most sets. They name the choice in 38–70% of the sets, but return both interpretations for the open request in only 15–58% of the sets they can compute. Accuracy hardly separates the models, but reporting does, and the larger model of each family does not close the gap. They almost never raise a choice that the wording already settles, so the gap is not caution against false alarms. Most missed interpretations are ones the same model computes correctly when asked for them alone, so the gap lies in reporting, not in computing. When a model settles a choice silently, it takes the interpretation that benchmark references take by convention, the interpretation benchmarks reward.

Finding 2: Benchmarks cannot see the gap (Section 6). When the wording of a BIRD or Spider question does not settle a choice, the reference SQL still takes one interpretation, by a convention of its author group. BIRD and Spider's own authors return one row when several tie and count joined rows, while the questions Spider imported from older datasets reverse one of these conventions each. The choice is not cosmetic. On BIRD, nearly one in four questions that leave a choice open changes its answer under the other interpretation, one in ten of all its development questions, and over two in five of the answers that BIRD marks wrong give that other interpretation, which the question allows.

Finding 3: Deciding first makes reporting reliable (Section 7). Phase 1 of our method, decide-then-answer, decides whether a request leaves a choice open, answers each interpretation of an open request in its own call, and lets execution decide what the user sees. Per 100 requests of which a fifth leave a choice open, it misses 7.8 fewer interpretations and shows 24.6 fewer wrong answers than plain prompting, at 2.6 calls per request. Sampling and DtP [26] show far more wrong answers, and the decide-then-ask behavior of production assistants leaves about half of all requests without an answer in that turn. A wide-search option keeps DtP's coverage

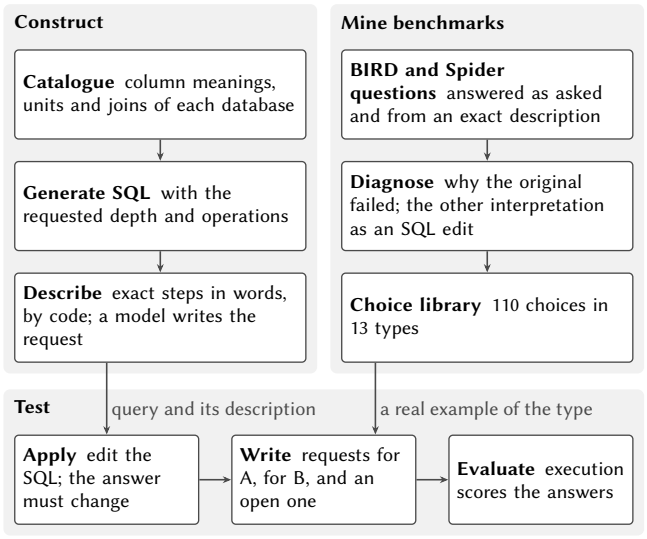


Figure 2: The SpecBridge pipeline.

with 60.9 fewer wrong answers. Phase 2 keeps the request and adds the user's pick as one clause, which beats rewriting it with DtP's or LangChain's prompts by 8.5 to 10.5 points.

In summary, we make three contributions: (1) request sets, controlled tests that change one calculation choice and are scored by execution, with a generator that builds them on real databases and a library of choices mined from benchmark failures (Sections 2–4); (2) findings on four models across our request sets, AMBROSIA and BIRD (Sections 5–6); and (3) a two-phase method that reports open choices and carries the user's pick, tested against prompting, sampling, DtP and production behavior rebuilt from documentation (Section 7). We release the generator, the 307 request sets with the authors' verdicts, all model responses, and a package that reproduces every number in this paper without new model calls; a supplement shows how common each choice type is, with examples, and gives the generator's details and the data behind each experiment, so every figure and table can be traced to its data file.

2 SPECBRIDGE OVERVIEW

SpecBridge is named for its central step, which bridges an exact specification of each query, written by code, and the natural requests a user would write. It turns a database into request sets in two lanes that meet in one step (Figure 2). The first lane generates queries whose computation is known exactly. The second lane mines, from benchmark failures, the choices that real requests leave open. The last step applies one such choice to a generated query and writes the three requests. We first follow two request sets through this process (Section 2.1) and then describe the steps of generating queries (Section 2.2), mining choices (Section 2.3), and building and checking request sets (Section 2.4), the step that keeps the three requests aligned on everything but the one choice.

2.1 Two examples

A request set fixes a database and every requirement of a query but one choice. The film request of Figure 1 asks for each rating's

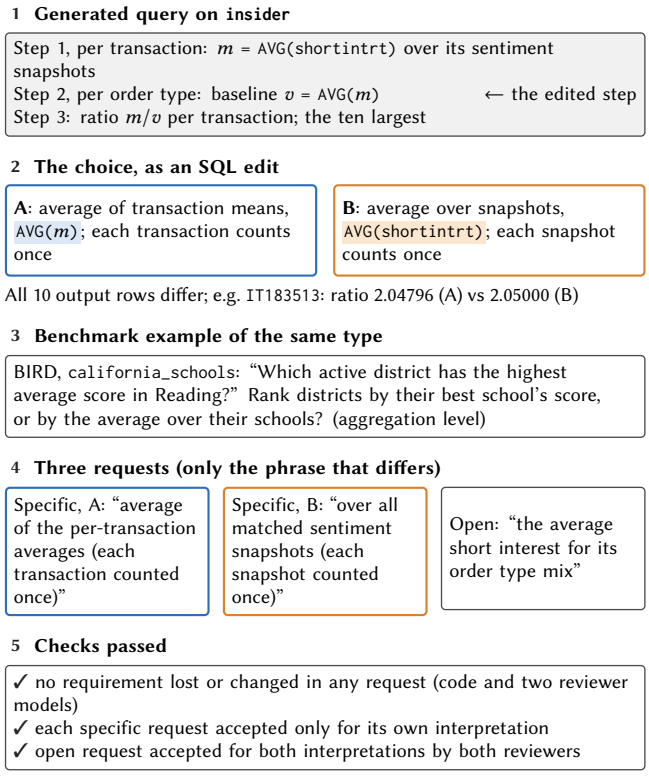


Figure 3: A request set on `insider`, from the generated query to its checks. Shaded: the SQL that differs.

average replacement cost and daily rental price, and compares that daily price with the average across ratings. Only the daily price is open:

$$A = \text{AVG}(\text{price}/\text{days}), \qquad B = \frac{\text{AVG}(\text{price})}{\text{AVG}(\text{days})}. \tag{1}$$

A averages each film's daily price; B divides two averages and so gives more weight to longer rentals. The specific request for A asks for the average of per-film daily prices, the one for B asks for the average price divided by the average duration, and the open request asks for the "average rental price per day." The rest of the wording is shared, and code checks that the two queries return different results on the database; a choice that does not change the answer is not tested there. The second example (Figure 3) is a three-step query on the `insider` database. It averages short-interest measurements for each transaction, compares each transaction with a baseline for its order type, and returns the ten largest ratios. The choice lies in the baseline. Interpretation A averages the transaction means, so each transaction counts once, while B averages the underlying snapshots, so a transaction with more snapshots weighs more. The open request asks for "the average short interest for its order type mix," and all ten output rows change between the interpretations.

2.2 Generating queries

Generation starts from the database, not from a benchmark, so it works on databases nobody has annotated. One model review of the schema and data first builds a catalogue of column meanings,

units and table relationships; after that, generation calls no model. A query starts from counts, sums or averages per entity, per group or overall. Later steps can compare a value with a broader average, take a share of a total, combine compatible quantities, aggregate again, or keep the top entities per group. We control two properties of each query, its arithmetic depth (the nesting of arithmetic expressions) and its step depth (the number of dependent query stages). Rules on column types decide which quantities can be combined or summed, and data checks reject empty answers, constant or redundant quantities, non-positive denominators and too-small groups. The generator looks ahead and drops a partial query as soon as it can no longer reach the requested depths, so no attempt is spent on a dead end, and every delivered query has exactly the arithmetic and step depths it was asked for.

Code then describes each query exactly in words. The text covers source rows, joins, unmatched rows, filters, groups, formulas, missing values, ordering, limits, ties and SQLite conventions, and the same SQL always gives the same text; we call it the exact description. Because the exact description states every requirement, a model that fails from it cannot compute the query, while a model that succeeds from it but fails from a natural request was misled by the wording. Section 2.3 builds on this contrast. Sonnet turns the exact description into one natural request using the column meanings. Claude Opus 5.5 (Opus below) checks the request for missing, changed or added requirements, and code checks how stored values are written. A failing request gets one revision with the reviewer's notes and is dropped if it fails again.

2.3 Mining choices from benchmark failures

The choices come from real benchmark failures. For each BIRD and Spider question, we compare a model's answer to the original question (with BIRD's evidence) with its answer to the exact description of the reference SQL. When only the description is answered correctly, the wording caused the failure, not the model's SQL, and we classify the reason as output columns, a different column or value, an unstated requirement, numerical precision, or an open choice. For an open choice, the other interpretation is written as an edit of the reference SQL, and both queries are executed. Opus and Sonnet judge each interpretation independently, and disagreements are kept. The library keeps choices of meaning, both open ones, where the wording fits both interpretations, and settled ones, where it selects one. For "What is the total number of molecules with double bonded oxygen?", both reviewers accept only the interpretation in which the oxygen atom is part of the double bond, although the reference SQL counts every molecule that contains an oxygen atom and a double bond. Figure 4 groups the library by choice type. The largest group of open choices concerns which column a phrase refers to, and half of the settled ones concern the counting unit.

The same choices appear in benchmark references, and the references do not agree on them. We call a question silent on a choice when neither its wording nor BIRD's evidence contains a word that settles it, such as "all" for ties or "distinct" for counts. For 12 of the 13 choice types, code reads which interpretation a reference takes, and Figure 4 counts the silent BIRD and Spider questions and the share whose reference takes the rarer interpretation. Every case of the rarer interpretation was reviewed and only confirmed cases are counted, so these shares are conservative. For example, 12.3% of silent tie questions return every tied row, and 8.6% of silent counts over a join count distinct entities. Here a benchmark that scores one interpretation treats a convention as the only answer.

2.4 Building request sets

A request set needs a choice that matters on the data, so code first reports where a generated task's columns have missing values, repeated join matches, distinct values or ties. Opus proposes one choice from the library and an exact SQL edit for it, and code checks that the edit applies, runs, changes the answer and can still be described exactly; a failed proposal gets one repair. Opus then writes the request set from a plan that lists the shared requirements and the one differing choice. It writes the request for A, edits it for B, and writes the open request. A library example of the same type may guide the wording, but never supplies the SQL. Opus and Sonnet, as two reviewers, check that the requests keep their requirements and judge each of the three requests against each interpretation. A set is kept only when both reviewers accept each specific request for its own interpretation alone and at least one accepts the open request for both interpretations. Five library types concern a filter, such as whether a cutoff is strict. Generated queries seldom filter, so for these types the generator adds one condition chosen from the data. Figure 4 gives an example of each type. The pipeline keeps 307 request sets on 12 databases, and 204 of them pass the authors' review and a check against real references (Section 4.2).

3 MEASURES

We score what a model computes by execution and what it says by a model judge. We report four measures: accuracy, both returned, named, and false alarms. A specific request selects one interpretation, and the open request fits both. A choice is settled when the wording selects one interpretation. We execute every returned query and compare its result with the reference result, with numerical tolerance and with rows tied at a cut-off treated as interchangeable; an execution error scores zero, as in Spider and BIRD scoring. For benchmark questions we also report a score that allows extra or reordered output columns.

For the open request, the model returns a main query and up to two alternatives. The model returns both interpretations when these queries include one with the result of A and one with the result of B. We report this share over all sets and over the sets in which the same model answers both specific requests correctly, where a miss cannot be a failure to compute. Written as a formula, with S_i the queries returned for the open request of set i, $e(p, \hat{p}) = 1$ when query $\hat{p}$ returns the result of reference p, and $B_i = 1$ when both specific requests are answered correctly,

$$\text{Both} = \frac{\sum_i B_i \prod_{j \in \{A,B\}} \max_{\hat{p} \in S_i} e(p_j, \hat{p})}{\sum_i B_i}. \tag{2}$$

A missed interpretation is computable when the same model answers the specific request for it correctly. A model names the choice when its explanation says that the choice is open, whether or not it gives SQL for the other interpretation. A judge model, which does not know which model answered, decides whether a response names the choice and whether its stated interpretation matches

Choice type	Example open phrase	One interpretation	Other interpretation
Aggregation level Rows or group averages?	“average standings points at races held there”	pool all records	average each race first
Strict cutoff Strict or inclusive?	“using 0.149 as the cutoff”	greater than 0.149	at least 0.149
Ties One tied row, or all?	“the top 10 telescopes by share of signals”	exactly ten rows	every telescope tied with the tenth
Counting unit Rows or entities?	“how many manager award winners”	distinct managers	award records
Missing values Keep or skip missing?	“group members by their major”	no major is its own group	members without a major left out
Referent column Which column is meant?	“schools in Riverside”	the district field	the county or the city field
Seven other types e.g. conditions on one record?			

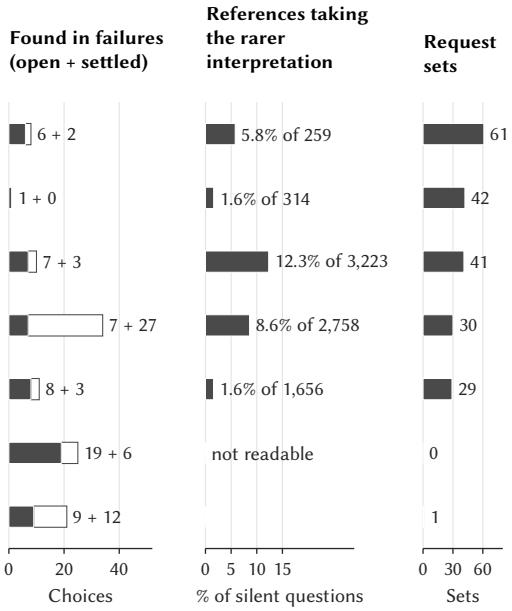


Figure 4: Choice types: an example open phrase with its two interpretations, choices found in benchmark failures (gray: open; white: settled), silent BIRD and Spider references that take the rarer interpretation, and accepted request sets.

Table 2: Four recorded responses and their scores.

Request, model	Main SQL	Alternative SQL	Names	Both	False alarm
Film, open, GPT-Sol	A: mean of daily prices	none	×	×	
Film, open, GPT-Luna	A: mean of daily prices	none	✓	×	
Awards, open, Sonnet	A: `COUNT(*)`	B: `COUNT(DISTINCT player_id)`	✓	✓	
Fees, specific, Haiku	A, as asked	B: mean of channel means			✓

its SQL. For a specific request, offering the other interpretation, or calling the choice open, is a false alarm. The measures check one another. Returning both interpretations alone would reward a model that adds every alternative it can think of, and the false alarms charge for that; naming alone would reward a warning without an answer, and returning both interpretations charges for the missing query. Table 2 scores four recorded responses.

On AMBROSIA, each ambiguous question comes with its interpretations, and an interpretation is found when some returned query gives its result. Recall is the share of interpretations found, and AllFound is the share of questions with every interpretation found [25]. Precision on an unambiguous question is the share of returned queries with its one correct result, so extras lower it.

4 EXPERIMENTAL SETUP AND TEST CHECKS

Sections 5–7 ask whether models report the choices they make, whether benchmarks see it, and how a system should report a choice and carry the user’s pick. This section gives the setup (Section 4.1) and checks that tests change only the choice (Section 4.2).

Table 3: The data and where each is used.

Use	Data	Sections
Request sets	12 databases, four each from Spider (baseball, Sakila, customers-and-invoices, Chinook), BIRD (financial, student club, thrombosis prediction, Formula 1) and LiveSQLBench (archeology, insider, crypto, alien)	2.4, 5, 7
Mining choices	481 BIRD development questions on five databases and 150 Spider development questions	2.3
Held-out test	100 BIRD and 50 Spider development questions, never used for mining	6
Conventions	every BIRD and Spider question silent on a choice, training and development	2.3, 6.1
Answer changes	every BIRD development question silent on a choice	6.2
Method	AMBROSIA test split, ambiguous and unambiguous questions	5, 7

4.1 Experiment settings

Data. Request sets use 12 databases, four each from Spider, BIRD development and LiveSQLBench[2]. The generated tasks request arithmetic depth 1–2 and step depth 3–6, and each database contributes 12 to 24 accepted request sets. Table 3 lists the data.

Models. We evaluate Haiku, Sonnet, GPT-Sol and GPT-Luna (`claude-haiku-4-5-20251001`, `claude-sonnet-5-5`, `gpt-6-sol`, `gpt-6-luna`). Opus writes the request sets, reviews them with Sonnet (Section 2.4), reviews the natural requests that Sonnet writes for generated tasks (Section 2.2), and judges the explanations; Opus is not evaluated. Models see the schema and documentation and have no file or tool access. By default a model answers in one call, with a prompt that asks for SQL, for a brief statement of “any consequential assumption that the request and the supplied context do not

[2]https://github.com/bird-bench/livesqlbench

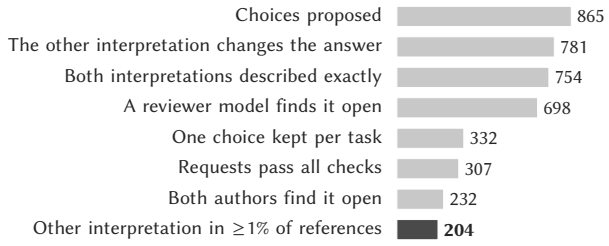


Figure 5: What each construction stage keeps, from proposed choices to accepted request sets.

settle," and for SQL of any other supported interpretation, without inventing alternatives. Computing is not our main question, so to test whether a model can compute an interpretation at all, we give it its best chance and let it explore. It may first run up to six rounds of five queries on the database, seeing twenty rows of each result. Exploration changes little: GPT-Sol solves 59 of 100 held-out BIRD questions without it and 61 with it, so one call is a fair default.

Statistics and baselines. Intervals resample tasks 2,000 times, keeping pairs together; proportions use Wilson intervals; paired comparisons use exact McNemar tests. For our method (Section 7), we compare with the AMBROSIA authors' prompt, the same prompt sampled three times, disambiguate-then-parse (DtP) [26], and the decide-then-ask of production assistants.

4.2 Checks on the tests

The findings rest on tests that change only the choice, so we check the tests first (Figure 5). Code confirms that in each of the 307 sets that pass all checks, the two queries differ in one step and the two specific requests differ only in the choice phrase. Two authors then cross-checked every set without seeing model results and kept it only if a reasonable analyst could mean either interpretation. They keep 232 of the 307 sets (76%). Most rejected sets have an open request whose wording already favours one interpretation, or a choice that convention settles, such as "between," which includes both ends in SQL and in almost every benchmark reference. We also keep only the choice types on which real references differ (Figure 4), which leaves 204 sets on aggregation level, cutoffs, ties, counting units, missing values and one negation. Every result below uses these 204 sets. The review does not drive the results. With no human filter, models return both interpretations in only 11–48% of the sets they can compute, so the gap holds without the review.

The generator itself is reliable. Its rules find usable queries on 152 of the 190 Spider, BIRD and LiveSQLBench databases we tried; the others lack the repeated rows the rules need. On the 12 evaluation databases it delivers 1,198 of 1,200 requested queries, all at the requested depths, and each of its rules is needed (Figure S2); without the look-ahead, almost no attempt reaches its target. Natural requests also lose little against exact descriptions (Figure S3). Sonnet and GPT-Sol lose about two points, and Haiku and GPT-Luna gain. Writing from a plan keeps the three requests aligned. On 60 tasks drawn at random across choice types, it keeps the shared requirements in 52 drafts, against 32 for direct generation ($p = 0.0002$), and passes all checks in 49 against 32 drafts.

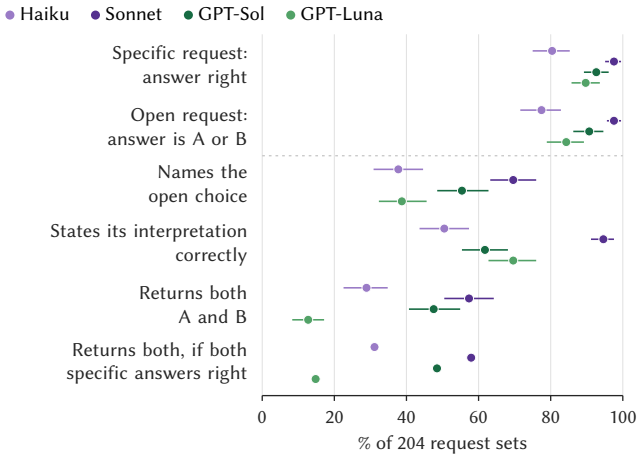


Figure 6: Answering and communicating on 204 request sets, per model and measure (95% intervals).

No model favours its own family. Claude models write and review the requests and Opus judges the explanations, so a Claude model might do better on them. We repeat each of these steps with GPT-Sol, from another family, and find no such advantage. When GPT-Sol writes the natural requests of 200 random generated tasks, neither GPT-Sol nor Sonnet gains on its own text. With exploration, GPT-Sol answers 395 of the 408 specific requests that Opus wrote, as many as Sonnet (396). When GPT-Sol rewrites the open requests of 60 random sets, no model returns both interpretations significantly more or less often (all $p \geq 0.18$). The Opus judge does not favour Claude models either. It credits naming the choice in 96–100% of responses that return both interpretations, and among responses that give one interpretation it credits GPT-Luna most often (27%).

5 MODELS MAKE CHOICES WITHOUT SAYING SO

A model that computes both interpretations when each is asked for usually returns only one of them for the open request, and often does not say that there was a choice. Asking for an explanation does not cause this gap. Against the same call asking only for SQL, correct answers to specific requests change by at most nine of 204 per model, and every paired interval includes zero. Nor does it make models raise settled choices: at most two of 204 responses to specific requests per model call the choice open.

Models return one interpretation where two are correct. Returning both interpretations is much rarer than returning one correct answer (Figure 6). Even among the sets in which a model answers both specific requests correctly, it returns both interpretations for the open request in only 58% for Sonnet, 48% for GPT-Sol, 31% for Haiku and 15% for GPT-Luna. Accuracy hardly separates the models, whose answers to specific requests are right in 80–98% of sets, but reporting does. The larger model of each family returns both about two and three times as often as the smaller one, yet none closes the gap. When a model states its interpretation, the statement matches its SQL in 99–100% of responses. The models describe one calculation correctly without saying that another, equally valid

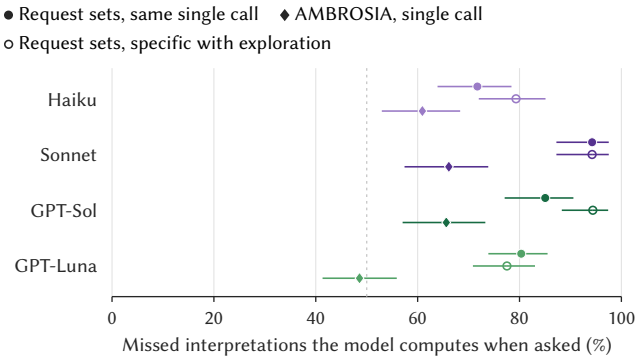


Figure 7: Missed interpretations that the same model computes when asked for each of them alone, per model, on request sets with one call or after exploring the database, and on AMBROSIA (Wilson 95% intervals).

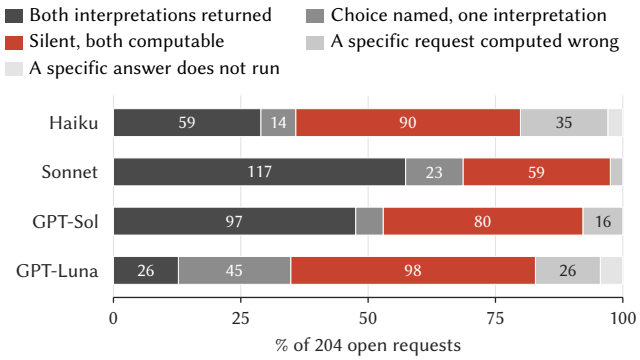


Figure 8: What each model does with the 204 open requests (one call with explanation). Computable: the same call answers both specific requests correctly.

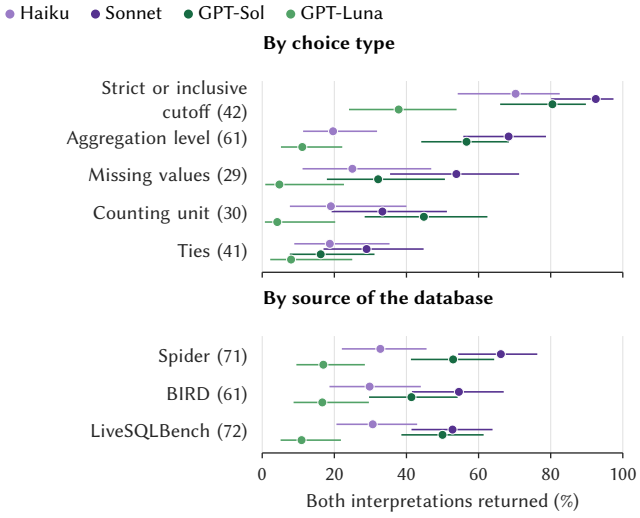


Figure 9: Open requests answered with both interpretations, by choice type and by source of the database, among the sets in which the same model answers both specific requests right (accepted sets in parentheses; Wilson 95% intervals).

calculation was possible on the same data. The unreported interpretation matters. In 98 of the 204 sets, the two interpretations return different rows, and in the other 106 the same rows with different computed values, so the user gets a different answer in every set.

Most misses are failures to report. Most interpretations a model misses on the open request are ones it can compute. The same model computes the missed interpretation correctly when asked for it alone in 72–94% of the missed sets (Figure 7). Figure 8 sorts every open request into five outcomes, namely both interpretations returned, the choice named but one interpretation given, one interpretation given silently although the model computes both, a specific request computed wrong, and a specific answer whose SQL does not run. For Haiku and GPT-Luna the most common outcome is the silent one, in which the model can compute both interpretations but gives one without naming the choice, and for Sonnet and GPT-Sol it is the second most common. A score that only checks whether both interpretations appear among the outputs, as top-k coverage [3] does, cannot tell these outcomes apart. Sampling does not surface the other interpretation either. On 55 randomly drawn sets, five SQL-only samples contain both interpretations in at most 14 for any model, while one explained response from Sonnet does so in 34. What is missing is a prompt to report the choice; SQL-only samples mostly repeat one interpretation.

The gap is widest for ties and counting units. Sonnet, the best model, returns both interpretations for 68% of aggregation-level sets but for only a third of tie and counting-unit sets, and the other models do worse (Figure 9). These choices hide in the SQL, in a `LIMIT` or a `COUNT` over a join, neither of which the request mentions. Cutoffs are the exception, because most cutoff requests give a value "as the cutoff" with no comparison word, which invites both the strict and the inclusive interpretation. The gap holds on every source of databases, and no model returns both interpretations for more than 67% of the sets from any source.

Naming a choice is not the same as giving the other interpretation. A model that says a choice is open often stops there. Among the sets in which a model names the choice and can compute both interpretations in the same call, it still returns only one interpretation in 10–65% of them, and GPT-Luna does so most often. Such a response warns the user but leaves them to write the other query, which is a step the model could have taken itself, since it computes that query when asked for it alone.

Silent choices follow benchmark conventions. When a model settles a choice without saying so, it takes the interpretation that benchmark authors take. In 61 tie and counting sets, one interpretation is the convention of BIRD and Spider references (Section 6.1), namely a plain `LIMIT` rather than every row tied at the cut-off, and counting joined rows rather than distinct entities. The conventional interpretation is the intended one in 29 of these sets, so the design favours neither. Yet when a model's main answer is one of the two interpretations, it is the conventional one in 76–91% of cases for every model (Figure 10). The conventional interpretation is also the simpler SQL, so these data cannot separate habit from simplicity. Either way, models settle an open choice the

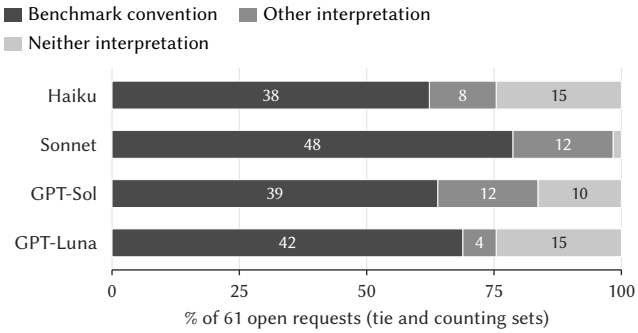


Figure 10: Main answer to the open request on 61 tie and counting sets: the interpretation BIRD and Spider references take by convention, the other interpretation, or neither.

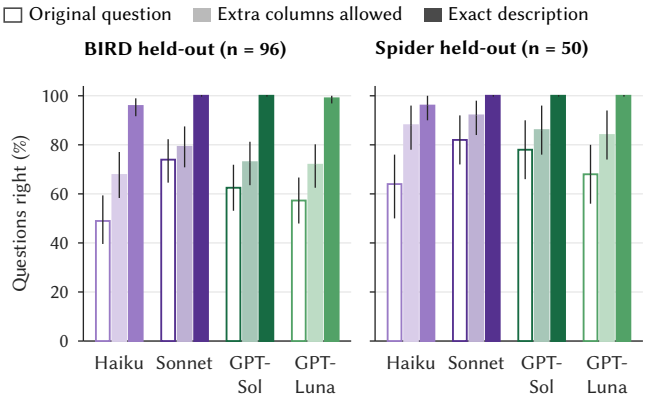


Figure 11: Held-out questions: original question versus the exact description of the reference SQL (95% intervals).

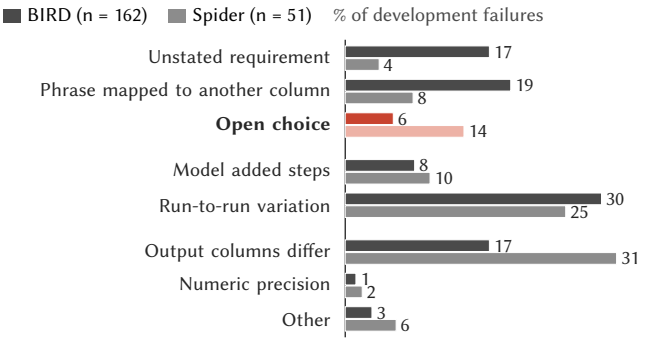


Figure 12: Why the original question failed, on development questions answered right from the exact description and wrong as worded, one cause per question.

way benchmark authors do, which is also the interpretation that a benchmark built by those authors rewards.

The same holds on AMBROSIA. The pattern is not an artifact of our tests. AMBROSIA pairs each ambiguous question with one unambiguous question per interpretation, and Saparina and Lapata [26] observe that models usually give one preferred interpretation. We split each missed interpretation into one the model cannot compute and one it computes but does not report. On 120 ambiguous test questions per model, the model computes 49–66% of the interpretations it misses when asked the unambiguous version (Figure 7). Even when a model answers every unambiguous version correctly, it returns all interpretations for the ambiguous question in only 12–28% of questions. On the 40 vagueness questions per model, which ask which column or value a phrase means, models compute 48–56% of the interpretations they miss, so the gap also covers the referent-column choices that our request sets leave out. On both benchmarks, models compute what is asked and stay quiet about settled choices, but leave open choices unreported, whatever the source of the ambiguity, wording or calculation.

Examples. For the film request of Figure 1, GPT-Sol states interpretation A, which matches its SQL, but treats the choice as settled; GPT-Luna says the choice is open but gives no second query; Sonnet adds B: "per-day price defined as the rating's average rental_rate divided by its average rental_duration (ratio of averages)." Responses can also mislead in the other direction. For a specific request on fees, Haiku computes a mean of channel means, which matches the other interpretation and contradicts the request; for an open count of awards, Sonnet names the choice and gives both `COUNT(*)` and `COUNT(DISTINCT player_id)`, the two counting units.

6 BENCHMARKS CANNOT SEE THE GAP

BIRD and Spider score a model against one interpretation even where a question leaves the choice open, and which interpretation they take depends on who wrote the question. On held-out questions, the four models answer 49–74% of BIRD questions correctly as worded but 96–100% from the exact description of the reference SQL (Figure 11). Allowing extra or reordered output columns closes only part of this gap, and on Spider the remaining gap is smaller.

The exact description gives the reference's steps, columns and choices at once, so this gap bounds what the wording costs rather than measuring ambiguity alone. To see what it consists of, Opus assigns one cause to each development question answered correctly from the description and wrongly as worded (Figure 12). Output columns, a phrase mapped to another column or value, unstated requirements and run-to-run variation cause most failures. A second labeler from another family, GPT-Sol, given the same evidence, agrees on the cause for 53% of the questions (Cohen's $\kappa = 0.44$); it often calls Opus's run-to-run variation output shape and finds an open choice in 35 questions where Opus finds 16, so the figure, if anything, undercounts open choices. Output columns are a matter of form, as the answer holds the right values in a different shape, such as a name instead of an ID. Open choices, which our library collects, are among them. Failures undercount them, because an answer following the benchmark's convention is scored right; Section 6.2 measures them on every silent question.

Other choices are about which column a word means. For "Name schools in Riverside…", filtering the district field returns six rows, the county field sixty and the city field seven. Code cannot tell from a reference which column the question's word could also mean, so request sets leave this type to AMBROSIA's vagueness questions.

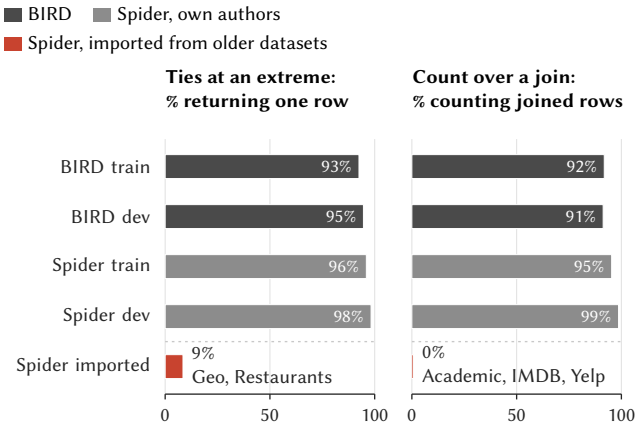


Figure 13: Share of silent questions whose reference takes the usual interpretation, by author group, for the two choices on which groups disagree, ties and counting.

Table 4: Six of the 17 BIRD questions on which all four models return the other interpretation, confirmed by the authors; BIRD scores each of these answers wrong.

Question	BIRD reference	Other interpretation
"Who is the tallest superhero?"	1 hero	2 tied heroes
"… cards in the set Coldsnap with the highest converted mana cost."	1 card	12 tied cards
"What are the expenses of the budget with the lowest remaining?"	1 expense	2 tied expenses
"How many of the banned cards are white border?"	258 rows	89 cards
"… normal glutamic pylvic transaminase level, how many of them are male?"	1,924 lab records	45 patients
"… diagnosed with SJS, how many of them have a normal level of total protein?"	780 lab records	37 patients

6.1 References follow unstated conventions

When a question is silent on a choice, its reference still takes one interpretation, and the interpretation depends on the author group. We read seven such choices off reference SQL, among them ties at an extreme, counting over a join, join type, and whether "between" includes its bounds. On five of the seven, every author group takes the same interpretation in at least 91% of silent questions. On ties and counting, the groups disagree (Figure 13). Asked for an extreme value, BIRD and Spider references return one row in over 90% of silent questions, but the Geo and Restaurants questions that Spider imported from older datasets return one row in 21 of 247. Asked for a count over a join, BIRD references count joined rows in most silent questions, and the imported Academic, IMDB and Yelp questions in none of 39. Each group keeps its convention in training and development data alike, so examples retrieved from a benchmark's training data can carry its authors' convention into a model's answers. For the same wording, a model that follows BIRD's convention is therefore scored right on BIRD and wrong on the imported Spider questions. The score measures agreement with the benchmark authors, not correctness.

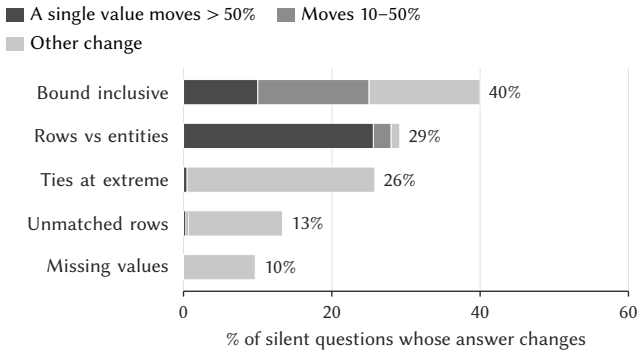


Figure 14: Silent BIRD development questions whose answer changes under the other interpretation, by choice type and by how far a single answer value moves.

6.2 The unstated interpretation changes answers

The choice a reference makes silently is not cosmetic. For each BIRD development question that is silent on a choice, we apply the library's edit for the other interpretation to the reference SQL wherever it gives a valid interpretation, and run both. Of 664 such questions, 156 (23%) change their answer (Figure 14). These are one in ten of all BIRD development questions. Counting changes the answer by the most, often by more than half of its value. For "How many accounts in North Bohemia has made a transaction with the partner's bank being AB?", the reference counts joined rows and returns 2,260, while counting each account once returns 55. Ties change about a quarter of answers, often the list of returned items, and join type and missing-value filters change about one in ten. The data behind these choices are common. Every one of the 12 evaluation databases has at least two kinds of data on which an unstated choice changes an answer, such as joins that repeat rows, columns with missing values and maxima shared by several rows.

BIRD scores the other interpretation as wrong. Accepting both interpretations raises accuracy from 25% to 57% on such questions; the difference is answers that BIRD counts as wrong SQL, although the question allows them. We ask the four models 137 BIRD development questions on which a tie, counting or join edit of the reference changes the answer, and score each answer against both queries. Two authors checked that the edit gives a valid interpretation of the question, and 79 questions pass. Of the 236 answers BIRD marks wrong on them, 99 return the other interpretation. On tie and counting questions, more answers match the edited reference (90) than the reference itself (37), and on 17 questions every model returns the other interpretation (Table 4).

7 A TWO-PHASE METHOD FOR OPEN CHOICES

A conversation about an open choice has two moments, before the user says which interpretation they mean and after. Production assistants stop to ask before (Section 1) and rewrite the request after (Section 7.3). Our method has one phase for each moment, and each phase rests on a finding of this paper: models compute

Request: "For each film rating, what is the average rental price per day?"

1 Phase 1: one call lists the interpretations, if any are open

A: average each film's daily price | B: average price / average duration

2 Each interpretation is answered in its own call

```
AVG(rental_rate /
    rental_duration)
```

```
AVG(rental_rate) /
AVG(rental_duration)
```

3 Both run on the database; identical results are shown once

G 0.668, PG 0.642, R 0.662 ≠ G 0.597, PG 0.600, R 0.616

4 The user sees one answer and the choice

Answer A. The request leaves open how the daily price is averaged; interpretation B gives G 0.597, PG 0.600, R 0.616.
Phase 2, after the user picks B: the request, unchanged, + "Divide the average price by the average duration."

Figure 15: Both phases on the film request. Blue and orange: interpretations A and B, as in Figure 1.

a missed interpretation once it is asked for on its own (Section 5), and a rewrite loses other requirements (Section 7.3).

7.1 Phase 1: decide, then answer

Phase 1, decide-then-answer, runs before the user replies (Figure 15). First, one call lists the interpretations of the request, each written as the full request with exactly one interpretation, which keeps the requirements that a short rewrite loses (Section 7.3); if it finds one, one more call answers it, and nothing else runs, so a clear request costs two calls and one query. Second, where a choice is open, each interpretation it lists, at most four, is answered in its own call, the setting in which models compute it. Third, every answer runs on the database, SQL that fails is dropped, and identical results are shown once. Execution can show that two interpretations differ on this data but not that they are equivalent [37], so a merged choice may matter on other data. The user sees the first answer with the others as alternatives, and phase 2 starts when the user picks one (Section 7.3). As an option for users who must not miss an interpretation, the second step can first search wide, with two more calls that list the interpretations and then any that are missing, using DtP's prompts [26]; we call this option decide-then-search.

7.2 Phase 1 against products and baselines

We developed the method on 60 AMBROSIA questions. The final test uses 150 new test questions per model (90 ambiguous, 60 unambiguous) and 111 randomly drawn request sets, with Sonnet and GPT-Sol. We compare it and its option with the AMBROSIA authors' prompt (plain), the same prompt sampled three times, DtP with its released prompts, and decide-then-ask, the behavior that Cortex Analyst and Genie document, rebuilt with our first call so only the action differs: a request it finds open gets a question and no answer, which these assistants give in their first turn.

Fewer misses and fewer wrong answers, at a small cost. Table 5 counts what a user gets per 100 requests of which a fifth leave a choice open. Pooled over both models, decide-then-answer misses 7.8 fewer interpretations than plain prompting ([2.2, 13.4])

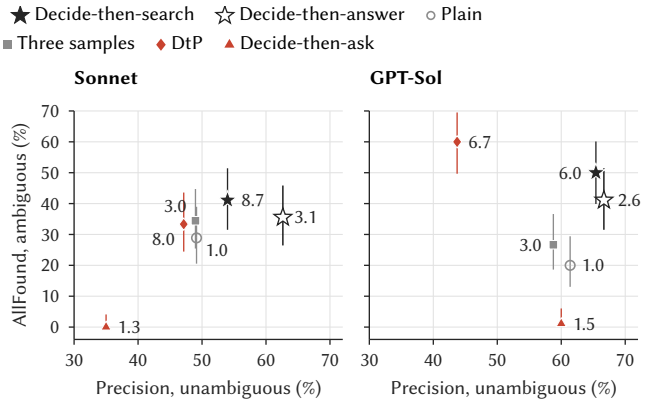


Figure 16: Methods on 150 new AMBROSIA test questions per model, with the calls per question beside each point (Wilson 95% confidence intervals).

Table 5: Decide-then-answer, its wide-search option (decide-then-search) and four baselines. AMBROSIA: per 100 requests of which 20% leave a choice open (lower is better). Request sets: 111 sets (higher is better).

		AMBROSIA, per 100 requests			Request sets	
Model	Method	Missed	Wrong shown	Calls	Both (of 111)	Specific (of 111)
Sonnet	Plain	35.6	108.2	100	64	109
	Three samples	28.0	144.4	300	72	108
	DtP	22.7	232.9	800	10	72
	Decide-then-ask	95.1	9.6	141	0	95
	Decide-then-answer	30.2	71.3	292	78	109
	wide search	26.2	206.4	763	79	109
GPT-Sol	Plain	48.4	58.7	100	47	92
	Three samples	42.7	90.7	300	60	92
	DtP	28.4	198.2	650	35	84
	Decide-then-ask	72.7	16.9	170	0	98
	Decide-then-answer	38.2	46.2	234	62	106
	wide search	35.6	102.9	450	68	106

and shows 24.6 fewer wrong answers ([9.7, 39.3]), at 2.6 calls per request; it is the only method better than plain on both counts. Three samples miss as many as it does and show 58.8 more wrong answers, at more calls. DtP misses 8.6 fewer interpretations but shows 156.8 more wrong answers, almost two per request, at nearly three times the calls. The order is the same when a tenth or half of the requests are open, and paired by question on AllFound, recall, precision, both and specific, decide-then-answer is better on 17 of 20 comparisons, losing only in coverage to DtP. On request sets, DtP's restatements lose other requirements, so decide-then-answer returns both interpretations for 42.8 points more open requests and is 26.6 points more often right on specific ones.

Decide-then-ask leaves half of all requests unanswered. Decide-then-ask stops on 70 and 49 of every 100 AMBROSIA questions and on 51 and 42 of every 100 requests, for Sonnet and GPT-Sol. Per 100 requests it leaves 49.7 more interpretations missing than decide-then-answer, and it also stops on clear requests, so it is 9.9 points less often right on specific ones. This measures what a user

has before replying; a clarifying dialogue may recover the choice only in a second turn, and only if the assistant noticed it.

Where the gain comes from. The first step decides where to answer. The wide-search option is DtP with the first step added, scored on the same saved answers, so their difference isolates that step: the same AllFound (−1.1 points), 60.9 fewer wrong answers per 100 requests and 14.3 points more precision. The gain is not from returning fewer alternatives. We cut DtP's answers, per question, to the number of interpretations that the first step lists. On the questions the first step chooses, this budget finds every interpretation for 7.6 points more ambiguous questions than the same budget spread over random questions ([3.8, 11.3]), and decide-then-answer, with the same budget, finds 15.0 points more than the cut DtP ([9.4, 20.6]) at no lower precision. Compared with decide-then-answer, the option misses 3.3 fewer interpretations per 100 requests but shows 95.9 more wrong answers at 2.3 times the calls, so it suits only users who must not miss an interpretation.

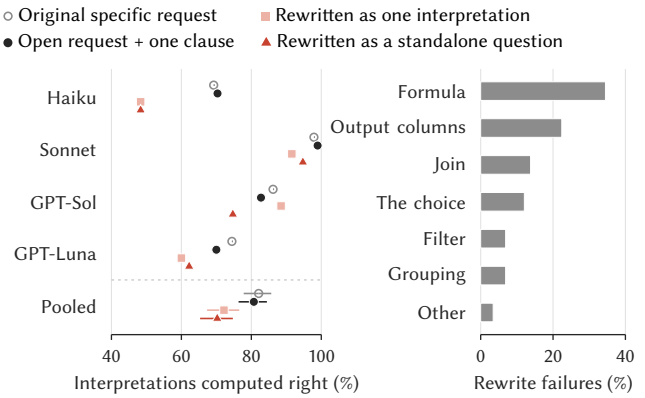


Figure 17: Editing versus rewriting a request (363 interpretations). Left: interpretations computed right from four request texts. Right: what the rewrite lost, by kind.

7.3 Phase 2: keep the request, add the pick

When the user picks an interpretation, today's systems rewrite the request. Cortex Analyst rewrites each follow-up into a self-contained question [33], the Fabric data agent rephrases each question [19], LangChain's ConversationalRetrievalChain [11] and LlamaIndex's CondenseQuestionChatEngine [18] condense each follow-up into a standalone question, and DtP writes each interpretation as a short question. Each rewrite is a new copy that can lose a requirement, so phase 2 keeps the request and adds the pick as one clause. On the financial database, one rewrite dropped the region, its share and the top-ten cut.

Request sets follow the opposite rule, as their three requests differ only in the choice. We test which rule a clarification should follow. For each interpretation of a request set, the same SQL prompt receives one of four texts: (a) the original specific request; (b) the open request rewritten by the model as one interpretation with DtP's prompt, told the intended interpretation; (c) the open request unchanged, followed by a one-sentence description of the interpretation; (d) the open request condensed by the model into a standalone question with LangChain's condense-question prompt [11], with the open request as the earlier turn and the interpretation as the follow-up. Texts (b) to (d) share the open request and description; we report 363 interpretations drawn at random from the 204 sets.

Keeping the request and adding one clause computes 80.7% of the interpretations correctly, as many as the original specific requests ($p = 0.57$, exact McNemar; Figure 17). Both rewrites compute fewer, 8.5 points fewer for the interpretation rewrite ($p = 0.0004$) and 10.5 for the standalone question ($p < 0.0001$), and the loss is largest for Haiku and GPT-Luna. The rewrite seldom loses the choice itself. Of the interpretations answered correctly from the original request and wrongly from the rewrite, the judge finds the choice lost in only 12%; the rest lose another requirement, most often a formula, an output column or a join (Figure 17, right).

The same holds on real questions. We repeat the comparison on 85 BIRD development questions where one library edit of the reference SQL changes the answer. A simulated user follows each question with one sentence that settles the choice, such as "If several rows tie at the cut-off, return all of them," and the edited reference is the gold answer. Over 170 question–model pairs with Sonnet and GPT-Sol, adding the clause beats the standalone question, 58.2% against 47.6%. As on our request sets, the rewrite keeps the clarified choice and loses something else; the gap is largest for ties.

8 IMPLICATIONS

Systems could answer open choices instead of asking. A model states its interpretation accurately and can usually compute the other one, yet seldom reports it. A system therefore need not wait for a stronger model: deciding first, answering each interpretation on its own and executing the answers already gives both interpretations in the turn in which an assistant would ask (Section 7).

Reporting could be a training target. The models we test already compute both interpretations of most requests; what they mostly lack is the habit of reporting the second one. This points to training rather than scale as a promising direction, since Sonnet, the strongest model, still misses the other interpretation in four of ten sets where it can compute it. Request sets could supply this signal. Each interpretation has its own executable reference, so a reward can credit an open answer only when it returns both interpretations and charge a specific answer for every alternative it adds.

Benchmarks could reward stating assumptions. Current benchmarks score a single answer per question, which suits questions with one correct interpretation. For a question that is silent on a choice, a complementary practice is to let the model state the assumption it made and the alternative it set aside, and to credit both when each matches its own reference. Request sets make this possible because each interpretation has its own reference, so a benchmark can report accuracy on specific requests, reporting on open requests, and restraint on settled ones. The library's SQL edit produces the second reference query from the first, as Section 6.2 does for BIRD, and could be accepted as a second answer wherever a question is silent, beginning with counting units and ties.

9 RELATED WORK

Natural language interfaces to databases. NaLIR [16] fixed "easy" ambiguities quietly and, for each "hard" one, explained its

interpretation and offered several for the user to choose from. Its ambiguities lie mainly in mapping words to schema elements and in how phrases attach. Our results show that the alternatives NaLIR asked users to choose among are now cheap to compute; what is missing is the report. Our method applies NaLIR's interaction to calculation choices and lets execution decide which are "hard".

Evaluation. Spider tests generalization across schemas [35]; BIRD adds large databases and external knowledge [17]; KaggleDBQA uses realistic databases and documentation [12]; Spider 2.0 tests enterprise workflows [14]. SParC and CoSQL add context and dialogue [34, 36], and BIRD-INTERACT tests clarification and recovery [10]. It scores whether a system completes a task after asking the user, whereas request sets score whether it reports a choice it made without asking. TrustSQL rewards a model for abstaining when it cannot answer reliably and penalizes wrong answers [13]; request sets reward a different behaviour, answering while reporting the open choice. Error detectors flag executable SQL that is wrong against one gold query [4], and verifiers turn such silent failures into abstentions [2]; a silent choice passes both, because its SQL is correct for one interpretation. Test suites [37] showed that execution on one database accepts wrong queries whose results match by chance, and distill databases that separate them from the gold query. Our concern is the opposite error, in which a correct query for another valid interpretation is rejected because the reference fixes one interpretation. Pourreza and Rafiei re-evaluated top models on Spider and BIRD by hand and found a perfect score out of reach "due to the multiple interpretations that can be derived from the provided samples" [24], and Wretblad et al. found noise in BIRD's questions and references whose correction changed which method ranks first [32]. We measure the first problem by construction rather than inspection and keep possible reference errors apart from open choices, because a wrong reference and a valid but different one need different fixes.

Ambiguous requests. The closest benchmarks are AmbiQT [3] and AMBROSIA [25] (Table 1). AmbiQT makes Spider questions ambiguous by changing the schema, with synonym columns or tables, split tables and precomputed aggregates, and checks whether both gold queries appear in the top five outputs. AMBROSIA has human-written questions with scope, attachment and vagueness ambiguity on generated databases. Both locate ambiguity in the schema or the wording. The choices behind the conventions of Section 6.1, such as ties, counting units, missing values and aggregation level, lie in the calculation, outside both taxonomies. Outside SQL, AmbigQA asks for every plausible answer to an ambiguous question [20], and conversational search studies when a system should ask a clarifying question [1]. Both start from ambiguity that the wording shows. In text-to-SQL the choice often lies in the calculation, where a model can answer fluently without noticing it, so request sets test reporting apart from computing.

Generation, SQL methods and semantic layers. DBPal [31] and GAZP [38] generate training pairs for a target database; DBPal fills seed templates from the schema and adds automatic paraphrases. We also start from the database, but for evaluation. RAT-SQL, PICARD, DIN-SQL, DAIL-SQL, CHASE-SQL and Alpha-SQL [7, 15, 22, 23, 27, 30] improve how the SQL is generated. Methods that retrieve training examples, such as DAIL-SQL, can carry the benchmark authors' conventions into the answers, so part of a benchmark gain may come from matching them. LookML has developers write each measure "once, in one place" [8], the dbt Semantic Layer centralizes metric definitions [6], and Cortex Analyst generates SQL from them [29]. A layer settles the choices its authors foresaw, usually a measure's formula and counting unit; per-request choices, such as ties or the rows a ratio includes, stay open, and these are the choices our method reports.

10 CONCLUSION

Text-to-SQL models make choices without saying so. Request sets, which change only one calculation choice, make this visible. Four models compute both interpretations of most sets when each is asked for, yet return both for the open request in only 15–58% of those sets, and when they settle a choice silently, they take the interpretation that benchmark references take by convention. Benchmarks cannot see the gap, because their references take one interpretation by conventions that differ by author group; over two in five answers BIRD marks wrong on such questions give the other interpretation. A method that decides first whether a choice is open and answers each interpretation on its own misses fewer interpretations than plain prompting, shows far fewer wrong answers than sampling and DtP, answers in the turn in which production assistants ask, and keeps the request when the user picks. The models already hold the other interpretation; what is missing is a step that shows it to the user, and a benchmark that asks for it and rewards it. Until then, a plausible number from a text-to-SQL system can answer a different question than the analyst asked, and nothing in the answer says so. We release the request sets, the library of choices and every model response, so that new models and choice types can be tested the same way and benchmarks can credit both interpretations of a question silent on a choice.

Limitations and future work. The generator covers a subset of analytical SQL, and the library comes from two benchmarks on SQLite, so other benchmarks may hold choice types we have not seen. The request sets cover every choice type on which real references differ, except quantifier, which the generator does not yet build, and referent column, which code cannot read off a reference; for the latter, AMBROSIA's vagueness questions show the same gap. Each request set tests one choice at a time, so requests that leave two choices open at once are not covered. The conventions are read by rules that see one SQL form each, so their counts are lower bounds. One model family wrote the request sets, but the results hold with another writer and without the human review (Section 4.2). The BIRD rescoring uses one call per question without exploration, so its shares may shift with stronger prompting. All four models are commercial. Decide-then-ask is rebuilt from the products' documentation, not run on the products. A model judge decides whether a response names the choice, but execution decides every accuracy score and whether both interpretations are returned. Other SQL dialects, multi-turn sessions and a user study of how analysts act on a reported choice are the next settings to test with request sets.

REFERENCES

[1] Mohammad Aliannejadi, Hamed Zamani, Fabio Crestani, and W. Bruce Croft. 2019. Asking Clarifying Questions in Open-Domain Information-Seeking Conversations. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, Paris, France, 475–484.

[2] Tarfah Alrashed, Madhup Sukoon, David R. Karger, and Natasha Noy. 2026. Developing and Benchmarking Verification Algorithms to Improve Text-to-SQL Generation. *Proceedings of the VLDB Endowment* 19, 11 (2026), 3732–3744. https://doi.org/10.14778/3836663.3836721

[3] Adithya Bhaskar, Tushar Tomar, Ashutosh Sathe, and Sunita Sarawagi. 2023. Benchmarking and Improving Text-to-SQL Generation under Ambiguity. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Singapore, 7053–7074. https://doi.org/10.18653/v1/2023.emnlp-main.436

[4] Shijie Chen, Ziru Chen, Huan Sun, and Yu Su. 2023. Error Detection for Text-to-SQL Semantic Parsing. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, Singapore, 11730–11743. https://doi.org/10.18653/v1/2023.findings-emnlp.785

[5] Databricks. 2025. Curate an Effective Genie Agent. Databricks documentation. https://docs.databricks.com/aws/en/genie/best-practices

[6] dbt Labs. 2025. dbt Semantic Layer. dbt documentation. https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl

[7] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2024. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *Proceedings of the VLDB Endowment* 17, 5 (2024), 1132–1145. https://doi.org/10.14778/3641204.3641221

[8] Google Cloud. 2025. Introduction to LookML. Looker documentation. https://docs.cloud.google.com/looker/docs/what-is-lookml

[9] Thomas Herndon, Michael Ash, and Robert Pollin. 2014. Does High Public Debt Consistently Stifle Economic Growth? A Critique of Reinhart and Rogoff. *Cambridge Journal of Economics* 38, 2 (2014), 257–279.

[10] Nan Huo, Xiaohan Xu, Jinyang Li, Per Jacobsson, Shipei Lin, Bowen Qin, Binyuan Hui, Xiaolong Li, Ge Qu, Shuzheng Si, Linheng Han, Edward Alexander, Xintong Zhu, Rui Qin, Ruihan Yu, Yiyao Jin, Feige Zhou, Weihao Zhong, Yun Chen, Hongyu Liu, Chenhao Ma, Fatma Ozcan, Yannis Papakonstantinou, and Reynold Cheng. 2026. BIRD-INTERACT: Re-imagining Text-to-SQL Evaluation via Lens of Dynamic Interactions. In *International Conference on Learning Representations*, C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust (Eds.), Vol. 2026. ICLR, Rio de Janeiro, Brazil, 44285–44333. https://proceedings.iclr.cc/paper_files/paper/2026/hash/496b549556509bbb9770bf9d335c5800-Abstract-Conference.html

[11] LangChain. 2024. ConversationalRetrievalChain and its condense-question prompt (CONDENSE_QUESTION_PROMPT). LangChain documentation and source, `langchain/chains/conversational_retrieval`. https://github.com/langchain-ai/langchain/blob/v0.3/libs/langchain/langchain/chains/conversational_retrieval/prompts.py

[12] Chia-Hsuan Lee, Oleksandr Polozov, and Matthew Richardson. 2021. KaggleDBQA: Realistic Evaluation of Text-to-SQL Parsers. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Association for Computational Linguistics, Online, 2261–2273. https://doi.org/10.18653/v1/2021.acl-long.176

[13] Gyubok Lee, Woosog Chay, Seonhee Cho, and Edward Choi. 2024. TrustSQL: Benchmarking Text-to-SQL Reliability with Penalty-Based Scoring. arXiv:2403.15879 https://arxiv.org/abs/2403.15879

[14] Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin SU, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, Victor Zhong, Caiming Xiong, Ruoxi Sun, Qian Liu, Sida Wang, and Tao Yu. 2025. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. In *International Conference on Learning Representations*, Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu (Eds.), Vol. 2025. ICLR, Singapore, 28691–28735. https://proceedings.iclr.cc/paper_files/paper/2025/hash/46c10f6c8ea5aa6f267bcdabcb123f97-Abstract-Conference.html

[15] Boyan Li, Jiayi Zhang, Ju Fan, Yanwei Xu, Chong Chen, Nan Tang, and Yuyu Luo. 2025. Alpha-SQL: Zero-Shot Text-to-SQL using Monte Carlo Tree Search. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research)*, Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (Eds.), Vol. 267. PMLR, Vancouver, Canada, 36810–36830. https://proceedings.mlr.press/v267/li25dt.html

[16] Fei Li and H. V. Jagadish. 2014. Constructing an Interactive Natural Language Interface for Relational Databases. *Proceedings of the VLDB Endowment* 8, 1 (2014), 73–84. https://www.vldb.org/pvldb/vol8/p73-li.pdf

[17] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., Red Hook, NY, USA, 42330–42357. https://doi.org/10.52202/075280-1835

[18] LlamaIndex. 2024. Chat Engine – Condense Question Mode (CondenseQuestionChatEngine). LlamaIndex documentation. https://developers.llamaindex.ai/python/examples/chat_engine/chat_engine_condense_question/

[19] Microsoft. 2026. Fabric Data Agent Creation. Microsoft Learn. https://learn.microsoft.com/en-us/fabric/data-science/concept-data-agent

[20] Sewon Min, Julian Michael, Hannaneh Hajishirzi, and Luke Zettlemoyer. 2020. AmbigQA: Answering Ambiguous Open-domain Questions. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Online, 5783–5797. https://doi.org/10.18653/v1/2020.emnlp-main.466

[21] Ziad Obermeyer, Brian Powers, Christine Vogeli, and Sendhil Mullainathan. 2019. Dissecting Racial Bias in an Algorithm Used to Manage the Health of Populations. *Science* 366, 6464 (2019), 447–453.

[22] Mohammadreza Pourreza, Hailong Li, Ruoxi Sun, Yeounoh Chung, Shayan Talaei, Gaurav Tarlok Kakkar, Yu Gan, Amin Saberi, Fatma Ozcan, and Sercan Arik. 2025. CHASE-SQL: Multi-Path Reasoning and Preference Optimized Candidate Selection in Text-to-SQL. In *International Conference on Learning Representations*, Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu (Eds.), Vol. 2025. ICLR, Singapore, 60385–60415. https://proceedings.iclr.cc/paper_files/paper/2025/hash/974ff7b5bf08dbf9400b5d599a39c77f-Abstract-Conference.html

[23] Mohammadreza Pourreza and Davood Rafiei. 2023. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., Red Hook, NY, USA, 36339–36348. https://doi.org/10.52202/075280-1577

[24] Mohammadreza Pourreza and Davood Rafiei. 2023. Evaluating Cross-Domain Text-to-SQL Models and Benchmarks. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 1601–1611. https://doi.org/10.18653/v1/2023.emnlp-main.99

[25] Irina Saparina and Mirella Lapata. 2024. AMBROSIA: A Benchmark for Parsing Ambiguous Questions into Database Queries. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., Red Hook, NY, USA, 90600–90628. https://doi.org/10.52202/079017-2876

[26] Irina Saparina and Mirella Lapata. 2025. Disambiguate First, Parse Later: Generating Interpretations for Ambiguity Resolution in Semantic Parsing. In *Findings of the Association for Computational Linguistics: ACL 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 16825–16839. https://doi.org/10.18653/v1/2025.findings-acl.863

[27] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 9895–9901. https://doi.org/10.18653/v1/2021.emnlp-main.779

[28] Nipun Sehrawat. 2024. Snowflake Cortex Analyst: Behind the Scenes. Snowflake Engineering Blog. https://www.snowflake.com/en/blog/engineering/snowflake-cortex-analyst-behind-the-scenes/

[29] Snowflake. 2025. Cortex Analyst. Snowflake documentation. https://docs.snowflake.com/en/user-guide/snowflake-cortex/cortex-analyst

[30] Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. 2020. RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Online, 7567–7578. https://doi.org/10.18653/v1/2020.acl-main.677

[31] Nathaniel Weir, Prasetya Utama, Alex Galakatos, Andrew Crotty, Amir Ilkhechi, Shekar Ramaswamy, Rohin Bhushan, Nadja Geisler, Benjamin Hättasch, Steffen Eger, Ugur Cetintemel, and Carsten Binnig. 2020. DBPal: A Fully Pluggable NL2SQL Training Pipeline. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 2347–2361. https://doi.org/10.1145/3318464.3380589

[32] Niklas Wretblad, Fredrik Riseby, Rahul Biswas, Amin Ahmadi, and Oskar Holmström. 2024. Understanding the Effects of Noise in Text-to-SQL: An Examination of the BIRD-Bench Benchmark. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 356–369. https://doi.org/10.18653/v1/2024.acl-short.34

[33] Aiwen Xu. 2024. Snowflake Cortex Analyst: Support for Multi-turn Conversation. Snowflake Engineering Blog. https://www.snowflake.com/en/blog/engineering/cortex-analyst-multi-turn-conversations-support/

[34] Tao Yu, Rui Zhang, Heyang Er, Suyi Li, Eric Xue, Bo Pang, Xi Victoria Lin, Yi Chern Tan, Tianze Shi, Zihan Li, Youxuan Jiang, Michihiro Yasunaga, Sungrok Shim, Tao Chen, Alexander Fabbri, Zifan Li, Luyao Chen, Yuwen Zhang, Shreya Dixit, Vincent Zhang, Caiming Xiong, Richard Socher, Walter Lasecki, and Dragomir Radev. 2019. CoSQL: A Conversational Text-to-SQL Challenge Towards Cross-Domain Natural Language Interfaces to Databases. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, Hong Kong, China, 1962–1979. https://doi.org/10.18653/v1/D19-1204

[35] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Brussels, Belgium, 3911–3921. https://doi.org/10.18653/v1/D18-1425

[36] Tao Yu, Rui Zhang, Michihiro Yasunaga, Yi Chern Tan, Xi Victoria Lin, Suyi Li, Heyang Er, Irene Li, Bo Pang, Tao Chen, Emily Ji, Shreya Dixit, David Proctor, Sungrok Shim, Jonathan Kraft, Vincent Zhang, Caiming Xiong, Richard Socher, and Dragomir Radev. 2019. SParC: Cross-Domain Semantic Parsing in Context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Florence, Italy, 4511–4523. https://doi.org/10.18653/v1/P19-1443

[37] Ruiqi Zhong, Tao Yu, and Dan Klein. 2020. Semantic Evaluation for Text-to-SQL with Distilled Test Suites. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Online, 396–411. https://doi.org/10.18653/v1/2020.emnlp-main.29

[38] Victor Zhong, Mike Lewis, Sida I. Wang, and Luke Zettlemoyer. 2020. Grounded Adaptation for Zero-shot Executable Semantic Parsing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Online, 6869–6882. https://doi.org/10.18653/v1/2020.emnlp-main.558